HW7: When Your Startup's Flash Sale Almost Failed

Group Report

Course: CS6650 — Building Scalable Distributed Systems **Date:** March 2026 **Team Members:**

- Sicheng Xue
- Yun Li
- Siwen Wu
- Yumeng Zeng

Individual Contributions

Member	Contribution
Sicheng Xue	Part II worker throughput analysis; Part III Lambda deployment and cold start observation
Yun Li	Part II queue depth and drain time measurement across all worker counts; Part III Lambda cold start and cost analysis
Siwen Wu	Part II async API performance and queue backlog measurement across all worker counts; Part III Lambda cold start and cost analysis
Yumeng Zeng	Part II sync vs async comparison, full worker scaling with queue depth/drain/CPU/memory; Part III Lambda cold start, memory, and cost analysis

---

Part II: The Simulated Problem — SQS + Async Processing

System Overview

All team members independently implemented:

- **Go HTTP service** with `/orders/sync` (buffered channel bottleneck, 3s payment simulation) and `/orders/async` (SNS publish, immediate 202 response)
- **Go SQS processor** with configurable worker goroutines (`WORKER_COUNT` env var)
- **AWS infrastructure** via Terraform: VPC, ALB, ECS Fargate, SNS, SQS
- **Load testing** via Locust: flash sale = 20 users, 10/sec spawn rate, 60 seconds (~57–60 orders/sec)

Phase 1 & 2: Sync Endpoint — Normal Ops vs Flash Sale

Yumeng Zeng's measurements:

Test	Users	Requests	Failures	Avg Response	Throughput
Normal ops (5 users)	5	38	0 (0%)	3,066ms	1.34 req/sec

Test	Users	Requests	Failures	Avg Response	Throughput
Flash sale (20 users)	20	95	0%*	**10,746ms**	1.66 req/sec

*No HTTP failures because Locust timeout was 30s; in production customers would abandon after 5–10s.

Bottleneck math:

- Payment processor: 1 order per 3 seconds
- With 5-slot semaphore: max throughput = 5 ÷ 3 = **1.67 orders/sec**
- Flash sale demand: **~57 orders/sec**
- Orders unable to be served: **~55.4/sec**

> Under flash sale load, response times exploded from 3s to 12s median. The synchronous system is ~34x undersized for peak demand.

Phase 3: Async Solution

Yumeng Zeng's measurements (20 users, 60s):

Metric	Sync	Async
Orders accepted	95	**1,550**
Acceptance rate	1.66/sec	**57/sec**
Avg response time	10,746ms	**53ms**
Failure rate	0%*	0%

Async accepted ~16x more orders with 202x faster response times.

The async endpoint publishes to SNS and returns 202 immediately — customers are never blocked waiting for payment processing.

Phase 4: Queue Buildup (1 Worker)

With 1 worker goroutine processing at 0.33 orders/sec against 57 orders/sec ingestion:

Comparative queue depth observations:

Member	Peak Queue Depth	Drain Behavior
Yun Li	~6,500–8,500	Did not drain within 15+ min
Yumeng Zeng	~5,225	Did not drain (~261 min estimated)

Queue growth rate: ~56.7 messages/sec **Time to drain 5,225 messages at 0.33/sec:** ~261 minutes

> CloudWatch `ApproximateNumberOfMessagesVisible` shows a steep spike during the 60s flash sale with virtually no drain — the single worker is overwhelmed by a factor of ~170x.

Phase 5: Worker Scaling

All members tested the same flash sale load while increasing worker goroutines within a single ECS task (256 CPU / 512MB RAM).

Theoretical Processing Throughput

Each worker processes one order per 3 seconds, so throughput scales linearly:

Workers	Theoretical Throughput	vs. Flash Sale Demand (~57/sec)
1	0.33/sec	0.6% of demand
5	1.67/sec	2.9% of demand
20	6.67/sec	11.7% of demand
100	33.3/sec	58.4% of demand
~171	**~57/sec**	**100% — break-even**

> Note: all members derived these numbers from the same formula (workers ÷ 3s). The meaningful comparison between members is in the **observed queue behavior** below — peak depth, drain time, and in-flight messages — which varied based on each member's test environment, prior backlog, and timing.

Queue Depth & Drain Time

Workers	Yun Li — Peak Queue	Yun Li — Drain	Siwen Wu — Peak Queue	Siwen Wu — RPS	Yumeng Zeng — Peak Queue	Yumeng Zeng — Drain
1	~6.5K–8.5K	Did not drain (15+ min)	~12,971	54.2	~5,225	~261 min (estimated)
5	~8,460	Did not drain (1+ hour)	~23,657	53.0	~10,204	~102 min (estimated)
20	~5,500	Drained to ~1.1K in ~1h	~12,879	53.4	~12,728	~31 min (estimated)
100	~11,400	Did not drain (prior backlog)	~3,191	52.3	~7,523	**~2 min (measured)**

> Siwen Wu also measured **In-Flight / Not Visible** messages: 1 worker (–7), 5 workers (44), 20 workers (143), 100 workers (491) — showing that more workers actively hold more messages in-flight simultaneously.

Resource Utilization (Yumeng Zeng)

Workers	CPU Utilization	Memory Utilization

Workers	CPU Utilization	Memory Utilization
1	~0%	~1.5%
5	~0%	~1.6%
20	~0%	~1.7%
100	**~12.7%**	~1.7%

CPU only became meaningful at 100 workers — confirming the bottleneck is I/O, not compute.

Key Finding

Minimum workers to prevent buildup at ~57 orders/sec:

```
required workers = ingestion rate × processing time = 57 × 3 = ~171
workers
```

- Yun Li observed: ~20 workers is the minimum to begin meaningful drain
- Siwen Wu observed: 100 workers reduced visible queue to ~3,200 (best result), with 491 messages actively in-flight
- Yumeng Zeng observed: 100 workers drained 7,523 messages down to 161 in ~2 minutes
- All members confirm that sub-100 worker counts cannot keep pace with flash sale load

---

Analysis Questions

Q1: How many times more orders did async accept vs sync?

From Yumeng Zeng's data: async accepted **~16x more orders** (1,550 vs 95) in the same 60-second window, at **202x lower latency** (53ms vs 10,746ms). The fundamental difference: sync ties up a server connection for the full 3-second payment processing; async acknowledges in <100ms and defers all work to background goroutines.

Q2: What causes queue buildup and how do you prevent it?

Queue buildup occurs when **ingestion rate > processing rate**. At 57 orders/sec with 3s/order, you need 171 concurrent workers minimum. Prevention strategies:

- **Static scaling:** Pre-provision ~180 workers for expected peak
- **Auto-scaling:** CloudWatch alarm on `ApproximateNumberOfMessagesVisible` → trigger ECS scaling when depth exceeds threshold
- **Serverless (Lambda):** AWS auto-scales concurrency automatically — no manual worker management (see Part III)

Q3: When would you choose sync vs async?

Use Sync when	Use Async when

Use Sync when	Use Async when
Client needs immediate result (fraud check, inventory)	Processing takes seconds+ (payment, fulfillment)
Latency SLA < 500ms	Client tolerates eventual completion
Operation is fast and predictable	Traffic is spiky and unpredictable
Simplicity matters more than throughput	Failures should retry silently

Part III: What If You Didn't Need Queues? — Lambda

Architecture Change

```
Before (Part II):  Order API → SNS → SQS → ECS Workers  (team manages
everything)
After  (Part III): Order API → SNS → Lambda              (AWS manages
everything)
```

Cold Start Observations

All members deployed a Go Lambda (`provided.al2`, 512MB) subscribed directly to the SNS topic and sent 5–10 test orders.

Member	Cold Start Init	Warm Duration	Overhead
Sicheng Xue	~70–74ms	~3,000ms	~2–3%
Yun Li	~71–72ms	~3,000ms	~2.4%
Siwen Wu	~70.70ms	~3,003ms	~2.35%
Yumeng Zeng	**77.63ms**	3,002ms	**2.6%**

Consistent finding across all members: Cold starts occur only on first invocation (or after ~5 minutes idle) and add 70–78ms overhead. On 3-second payment processing, this is **negligible (2–3%)**.

Yumeng Zeng additionally observed: actual memory used = **20MB out of 512MB allocated** (3.9%) — Lambda is significantly over-provisioned at 512MB for this workload; 128MB would likely suffice.

Cost Analysis

Scenario	ECS Workers	Lambda
Fixed monthly cost	**~$17/month** (always running)	**$0** (pay per invocation)
10K orders/month	$17	$0 (free tier)
267K orders/month	$17	~$0 (free tier limit)
Break-even	—	~1.7M orders/month

Should Your Startup Switch to Lambda?

Team consensus: Yes, for early-stage startups — with caveats.

The 2–3% cold start overhead is negligible on 3-second payment processing. Lambda eliminates the entire operational layer that caused the "3am alerts" described in the assignment: no SQS queue depth monitoring, no manual worker scaling, no ECS task health management, no visibility timeout tuning.

The key trade-off is durability: SNS retries a Lambda failure only twice before discarding the message, whereas SQS supports configurable retries and dead-letter queues. For a startup under 267K orders/month, this is acceptable — lost orders are rare and customers can retry. The right time to reintroduce SQS is when:

1. Monthly volume exceeds 267K (Lambda cost approaches ECS cost), **or**
2. Payment failure/retry guarantees become a hard business requirement

Until then, Lambda is strictly better: lower cost, zero operational burden, and automatic scaling to any load.

Siwen Wu's perspective: Lambda successfully processed SNS-triggered orders with the same 3-second logic. Cold start overhead was very small relative to total execution time. However, SQS + ECS provided stronger queue buffering and control — Lambda is the right choice for simplicity and cost, but SQS should be reintroduced when durability guarantees become critical.

Summary

Phase	Key Finding
Sync bottleneck	1.67 orders/sec max throughput vs 57 orders/sec demand — 34x gap
Async benefit	16x more orders accepted, 202x faster response
Queue buildup	Grows at ~57 msg/sec with 1 worker — takes 261 min to drain
Worker scaling	Linear throughput gain; ~171 workers needed to match flash sale
Lambda	Same performance, $0 cost, zero operational overhead for startups